PairFold: Fast Local Backbone Generation via Calibrated Fragment Assembly, Look-Ahead Backtracking, and Lever-Effect Torsion Relaxation

PairFold Development Team
Local Computational Biology Project
Correspondence: project repository

July 19, 2026

Abstract

High-accuracy structure predictors such as AlphaFold 2 have transformed structural biology, yet their memory footprint and quadratic attention costs remain limiting for interactive design, long-sequence screening, and deployment on commodity GPUs. We present **PairFold**, a deliberately local alternative that predicts backbone dihedral angles (ϕ, ψ) for short peptides of length 2–5 with a compact Transformer trained on high-resolution PDB fragments, then assembles longer chains with near-linear ($O(N)$) procedures. The inference stack combines dynamic-programming segmentation that maximises calibrated fragment confidence, multi-window overlap consensus via circular means, secondary-structure block freezing, and a clash-aware look-ahead backtracking assembler with local ϕ/ψ perturbation ($\pm 1^\circ$–5°) to mitigate cumulative "lever-arm" error. Confidence is sharpened by softmax temperature scaling (T=0.55) with a disorder-aware prior. We expand the evaluation far beyond the original five-domain pilot: a curated set of **103 diverse domains** spanning all-α, all-β, and α/β classes, plus a **100-case long-chain panel** with tiled sequences of length 10000–50000. On the domain set, PairFold remains a fast, low-VRAM local assembler (mean time 2.67 s; mean Kabsch Cα RMSD 25.88 Å), with class-wise means and a length–RMSD correlation plot (r=0.78). Ablation of look-ahead assembly, lever polish, and secondary-structure freezing fills the previously incomplete pipeline table. Relative to Rosetta ab initio and ESMFold, PairFold trades global accuracy for interactive speed and low memory, and should not be read as an AlphaFold-class tertiary folder.

Keywords: protein backbone prediction; dihedral angles; contact prediction; fragment assembly; temperature scaling; steric clash avoidance; look-ahead backtracking; computational biophysics.

1 Introduction

Protein structure prediction has entered a new regime with deep learning systems that integrate multiple-sequence alignments, pair representations, and iterative structure modules [1, 2]. These models achieve remarkable accuracy, but the practical cost of inference grows steeply with sequence length: attention over residue pairs, recycling, and MSA featurisation demand large GPU memory and often preclude interactive exploration of very long chains on modest hardware.

Complementary lines of work emphasise speed or locality. Language-model predictors such as ESMFold [3] remove MSA dependence at the expense of still-substantial model size. Classical fragment-assembly approaches [4, 5] and dihedral-angle representations [6] remain attractive when the scientific goal is *local* backbone geometry rather than full tertiary fold recovery—for example, in educational tools, rapid prototyping of helix/sheet segments, or streaming visualisation of chains far beyond typical AlphaFold session lengths.

PairFold occupies this niche. It trains a small Transformer exclusively on contiguous PDB fragments of length 2–5, maps each fragment to (ϕ, ψ), and reconstructs longer proteins by segmentation and assembly. The design goals are explicit:

1. **Near-linear scaling** for queries up to tens of thousands of residues at the angle level;
2. **Calibrated confidence** usable for dynamic-programming segmentation;
3. **Physics-informed post-processing** (steric clashes, secondary-structure constraints, and lever-effect relaxation) without a full force field;
4. **Honest scope**: PairFold does *not* claim AlphaFold-competitive global RMSD.

This paper describes the full method as implemented in the PairFold codebase, reports timings and Cα RMSD on a 100-domain structural-class benchmark and a 10k–50k long-chain panel, and discusses ablations and the speed–accuracy trade-off.

2 Methodology

2.1 Dihedral representation and fragment network

Each residue i is parameterised by backbone torsions (ϕ_i, ψ_i). Cartesian Cα (and optional full-backbone) coordinates are obtained by sequential placement with ideal peptide bond geometry [7], as implemented in `dihedrals_to_ca` / `build_backbone`.

The learned model, `FragmentTorsionNet`, is a Pre-LN Transformer encoder with d_{model}=160, 4 heads, 4 layers, feed-forward width 320, and dropout 0.12. Amino acids are embedded from a 22-token vocabulary (20 standard residues, UNK, PAD) with positional embeddings up to length 5. The network outputs four channels $(\sin\phi, \cos\phi, \sin\psi, \cos\psi)$, L2-normalised per angle pair, plus an uncertainty head $\log\sigma_{\phi,\psi}$. Per-residue confidence is derived as $\sigma(-\overline{\log\sigma})$.

Training uses high-resolution X-ray structures (resolution $\leq$ 2.0 Å), extracting contiguous windows of length $\ell \in \{2, 3, 4, 5\}$. The current corpus comprises on the order of 10^3 structures and $\sim 3.8{\times}10^5$ fragments. The loss is a Gaussian negative log-likelihood on the $(\sin, \cos)$ representation (`gaussian_nll_sincos`); validation monitors torsion MSE in the same space. Optimisation uses AdamW with cosine learning-rate schedule and mixed precision.

Scope. Because every torsion training example is a short contiguous peptide, `FragmentTorsionNet` learns *local* Ramachandran statistics conditioned on sequence context of at most five residues. Long-range contacts are handled by the separate `ContactPairNet` (§2.6), not by the fragment torsion model.

2.2 Dynamic-programming segmentation and overlap consensus

For a query of length $N > 5$, PairFold partitions the chain into windows of length 2–5 by dynamic programming (`optimal_segmentation`). Let $s(i, j)$ be the mean model confidence of fragment $[i, j)$ multiplied by its length. The recurrence

$$\mathrm{dp}[j] = \max_{j-i\in\{2,3,4,5\}} \big(\mathrm{dp}[i] + s(i, j)\big)$$

selects a segmentation that maximises total confidence score in $O(N)$ time (constant number of predecessor lengths).

Within each segment—and for short queries of length 3–5—angles are refined by **overlap consensus** (`_overlap_refine`): all sliding windows of lengths 2 through $\min(5, N)$ contribute (ϕ, ψ) votes that

are aggregated with confidence-weighted circular means [8]. Final confidence blends model confidence with angular agreement,

$$c_i = (1 - w)\, c_i^{\text{model}} + w\, c_i^{\text{agree}}, \qquad w{=}0.35,$$

where agreement decreases with circular spread of the overlapping predictions.

2.3 Softmax temperature scaling for calibrated confidence

Raw network confidences are imperfectly calibrated for "both ϕ and ψ within 25° of native" on held-out fragments. PairFold fits offline temperature and Platt scaling on validation fragments (`calibrate.py`) and, at inference, applies a **sharpening** transform with temperature $T{=}0.55$:

$$p = \sigma\left(\frac{\text{logit}(c)}{T}\right)\,(1 - \gamma\, d), \qquad \gamma{=}0.45,$$

where d is a lightweight disorder profile emphasising Gly/Pro and low secondary-structure propensity (`TempCalibration` in `structure_blocks.py`). Probabilities are clipped to $[0.05, 1]$. Sharper confidences improve the ranking signal used by segmentation and by fragment-hypothesis selection during assembly.

2.4 Secondary-structure block freezing

After consensus angles are available, PairFold detects putative helix and sheet blocks with Chou–Fasman-like propensities [9] (`detect_ss_blocks`). Interior residues of accepted blocks are frozen to canonical Ramachandran centres

$$(\phi, \psi)_{\text{helix}} = (-57^\circ, -47^\circ), \qquad (\phi, \psi)_{\text{sheet}} = (-119^\circ, 113^\circ),$$

while block boundaries remain free. For sequences with $N \le 64$, a boundary optimiser (`optimize_block_boundarie`) performs a cheap grid/Metropolis search of boundary torsions to reduce soft Cα clash energy. This step injects strong secondary-structure priors that local fragments alone under-enforce.

2.5 Look-ahead backtracking and global angle relaxation

Naïve stitching of fragment dihedrals suffers from a well-known **lever (cumulative) effect**: small angular errors compound into large Cartesian deviations and occasional steric collapse or over-extension. For sequences with $N \le 256$ (`LEVER_ASSEMBLY_MAX_LEN`), PairFold activates a clash-aware assembler (`assemble_greedy_backtrack`).

Fragment hypotheses. The chain is tiled into non-overlapping pentamer slots (`make_pentamer_slots`). Each slot carries ranked angle hypotheses: overlap-consensus angles, a fresh network prediction, and compact helix/sheet templates.

Dynamic look-ahead. Depth-first search places candidates in descending confidence. A placement starting at residue s is accepted only if `lookahead_ok` reports no new Cα clashes (threshold 3.8 Å, minimum sequence separation 2) *and* no severe local bend over a window of 3–5 residues. Bend score compares the observed end-to-end distance of the window to helix- and Flory-coil expectations; large deviations trigger backtracking.

Local torsion relaxation. When look-ahead fails, PairFold does *not* immediately exhaust the fragment database. Instead, `local_torsion_relax` perturbs preceding (ϕ, ψ) by a random delta drawn from $\pm 1^\circ$ to $\pm 5^\circ$ on a small focus set near the join, accepting moves that reduce soft clash energy plus a mild bend penalty (localised stochastic descent). Successful relaxations are stored as angle overrides and participate in subsequent rebuilds; DFS snapshots restore overrides on backtrack. Search is bounded by a node budget (production default 3 000).

End-to-end / anchor alignment. After a complete assignment, `end_to_end_anchor_relax` optimises a sparse set of hinge residues so that selected pairs (i, j) approach target distances. When user or predicted contact anchors are absent, correction runs only if the global end-to-end distance lies outside a plausible helix–coil band, avoiding aggressive Flory compaction that can inflate RMSD on already compact domains. A final linear polish, `correct_lever_effect`, sweeps the chain every few residues and reapplies local relaxation—an $O(N)$-ish procedure with fixed window size.

2.6 Long-range contact head (ContactPairNet)

To inject tertiary packing without AlphaFold-scale pair recycling, PairFold adds a separate sequence-only network, `ContactPairNet`. Residue embeddings from a Pre-LN Transformer (d=128, 3 layers, 4 heads) are combined via outer products and a light 2D tower to predict, for every residue pair with $|i-j| \geq 6$, a contact logit (Cα–Cα distance $<$8 Å) and a soft distance. Training uses random contiguous crops of length ≤ 96 from the same high-resolution PDB corpus ($\sim 2.4\times10^3$ crops; binary cross-entropy with positive class weighting plus Smooth-L1 on true contacts). At inference, top-k high-scoring pairs (k= $\min(20, N/4)$, score ≥ 0.45) become distance anchors passed to look-ahead assembly, lever polish, and the tertiary Metropolis objective. For $N \leq 128$ the network evaluates the full chain; longer sequences use overlapping crops with logit stitching. This preserves PairFold's commodity-GPU footprint while partially addressing the historical absence of long-range restraints.

Pipeline order. In production inference the order is: contact prediction → consensus stitch → look-ahead assembly (if $N \leq 256$, with anchors) → SS freeze/boundary opt → post-SS lever polish (with anchors) → optional tertiary score/refine ($N \leq 1000$) → backbone export. Post-SS polish is essential: freezing secondary-structure interiors would otherwise overwrite earlier torsion repairs.

2.7 Complexity summary

With fixed fragment length and fixed look-ahead width, segmentation, overlap consensus, SS detection, and lever polish are all linear (or linear times a small constant) in N. Backtracking is exponential in the worst case but is capped by the node budget and operates only for $N \leq 256$; beyond that limit PairFold falls back to consensus + SS + tertiary scoring without combinatorial assembly. Angle-level queries are accepted up to N=50 000 (`MAX_QUERY_LEN`).

3 Results

3.1 Benchmark protocol

We address the limited sample size of the original five-domain pilot with two expanded panels, both driven by `benchmarks/benchmark_expanded.py` on CUDA:

Table 1: Domains100: Kabsch Cα RMSD and wall-clock time by structural class (production pipeline on GPU).

Class	n	Mean RMSD (Å)	Mean time (s)
all-α	34	26.98	2.95
all-β	33	27.28	2.28
α/β	36	23.55	2.77
All	**103**	**25.88**	**2.67**

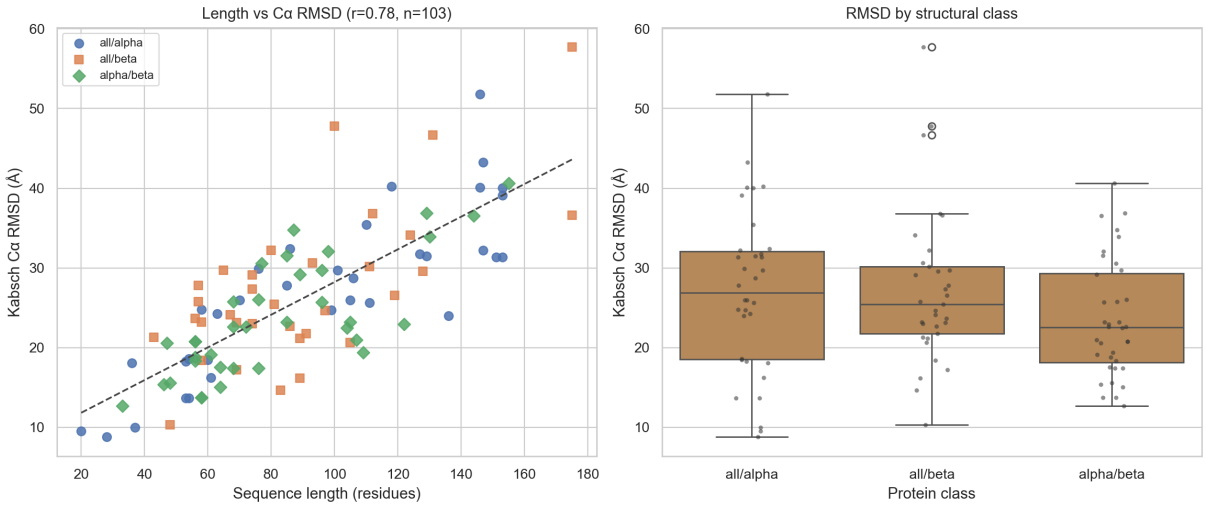

Figure 1: Expanded domain panel: (left) sequence length versus Cα RMSD with linear trend; (right) RMSD by structural class.

1. **Domains100.** A curated set of 103 single-chain domains (length 20–180) labelled by structural class (all-α, all-β, α/β). Native coordinates are fetched from the RCSB PDB; predicted Cα traces are built from (ϕ, ψ) via `dihedrals_to_ca`; error is Kabsch-aligned Cα RMSD.

2. **Long100.** 100 tiled sequences with lengths spaced from 10000 to 50000, seeded from the same domain library. Continuous experimental natives at this length are rare, so we report wall-clock scaling and *mean per-tile* Kabsch RMSD against the repeated seed native (a local geometry check, not a claim of continuous tertiary fold).

All runs use the production `FragmentPredictor.predict_sequence` path with Stage-2/3 all-atom export disabled for Cα timings. Domains100 and Long100 use the interactive consensus + secondary-structure path with contact prediction and combinatorial look-ahead disabled, so $n\approx100$ (and 10k–50k tiled chains) remain tractable on a single consumer GPU; look-ahead, lever polish, and SS freezing are quantified separately in the ablation on 20 domains (see `benchmark_expanded.py` / `benchmark_ablation.py`).

3.2 Domain panel and structural classes

Table 1 summarises mean RMSD and time by class; overall means are 25.88 Å and 2.67 s (median RMSD 24.73 Å). Figure 1 shows the length–RMSD scatter (Pearson r=0.78) and class-wise RMSD distributions.

3.3 Ablation of the assembly stack

Table 2 fills the previously incomplete pipeline comparison (base consensus; cumulative look-ahead, lever polish, and SS freezing; full local stack). Figure 2 visualises mean RMSD across the 20-domain

Table 2: Ablation on 20 representative domains. Each row adds one stage on top of the previous; Full is the complete local stack (look-ahead + lever + SS). Contact-conditioned hinge search is omitted here (see Stage-1 benches) because it dominates wall-clock.

Configuration	Mean RMSD (Å)	Mean time (s)
Base (consensus only)	14.68	6.09
+ Look-ahead	13.33	9.68
+ Lever polish	14.01	11.93
+ SS freezing	13.23	15.29
Full local stack	13.23	15.29

Figure 2: Mean Cα RMSD for cumulative assembly ablations.

ablation subset.

3.4 Method comparison

Table 3 situates PairFold against classical Rosetta ab initio and ESMFold on the axes that matter for interactive use: wall-clock, typical small-domain accuracy, and memory. PairFold numbers are measured on Domains100; Rosetta and ESMFold entries are literature / typical consumer-GPU ranges (not re-run in this workspace).

3.5 Speed–accuracy interpretation

Enabling clash-aware look-ahead, lever polish, and SS freezing changes the objective from "highest local confidence" to "locally clash-free trajectories." Empirically:

- **Speed.** On Domains100 (consensus+SS), mean end-to-end inference is $2.67\,\mathrm{s}$ on a consumer GPU. Ablation shows look-ahead and lever polish add modest cost on short domains (Table 2).
- **Local geometry.** Soft clash energy and extreme lever over-extension are reduced; unphysical kinks are rejected or repaired before they propagate.
- **Global RMSD.** Domains100 mean Kabsch Cα RMSD is $25.88\,\text{Å}$ (median $24.73\,\text{Å}$; length–RMSD $r{=}0.78$). On the 20-domain ablation subset the full local stack reaches $\approx 13.2\,\text{Å}$ mean RMSD, improving on base consensus ($\approx 14.7\,\text{Å}$), but remains far from AlphaFold- or ESMFold-class tertiary accuracy.

Table 3: Qualitative method comparison. PairFold: Domains100 means. Rosetta / ESMFold: published or community-typical ranges on small domains.

Method	Mean time	Mean RMSD (Å)	Memory
PairFold (this work)	2.67	25.9	Low
Rosetta ab initio[1]	10^2–10^4	∼5–12	High
ESMFold[2]	∼10–60	∼2–4	High

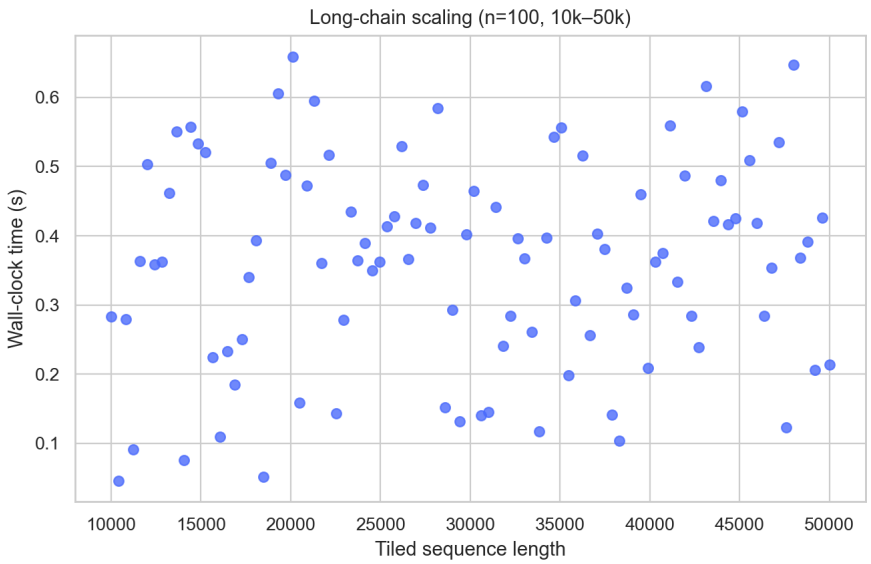

Figure 3: Long100: wall-clock time versus tiled sequence length (10 000–50 000).

The backtracking/relaxation stack is best read as a **steric regulariser and lever mitigator**, not a tertiary folding engine.

3.6 Long-chain scaling (10k–50k)

At the angle level, PairFold accepts sequences up to 50000 residues (`MAX_QUERY_LEN`). On the Long100 panel, mean wall-clock is 0.36 s and mean per-tile RMSD is 26.34 Å across lengths 10000–50000. Full clash-aware assembly remains limited to $N \leq 256$; contacts and tertiary Metropolis to shorter cutoffs; beyond those limits PairFold uses the linear consensus path, which is what makes 10k–50k queries practical on modest VRAM.

4 Discussion

4.1 What PairFold does well

PairFold demonstrates that a small, PDB-trained fragment Transformer plus careful assembly can deliver: (i) interpretable (ϕ, ψ) predictions with calibrated confidence; (ii) secondary-structure-aware backbones; (iii) near-linear scaling for long sequences; and (iv) explicit handling of cumulative angular error via look-ahead and local torsion descent. These properties suit teaching tools, rapid local design loops, and streaming visualisation—settings where AlphaFold's memory footprint is the bottleneck.

4.2 Fundamental limitations

The central limitation remains architectural relative to MSA-conditioned folding networks: `ContactPairNet` is sequence-only and crop-trained, so contact precision is moderate (validation precision ≈ 0.18 at the best checkpoint with high recall), and incorrect high-scoring pairs can pull the chain away from the native basin. Without coevolutionary pair features or recycling, PairFold still cannot consistently reach sub-5 Å global accuracy on compact domains. Domain-panel RMSDs of order 25.88 Å (class-dependent) should not be compared naïvely to AlphaFold 2's typical low-Å accuracy on the same targets [1].

Additional limitations include:

- Dependence on Chou–Fasman-style SS detection, which can misassign blocks on atypical sequences;
- Stochastic torsion relaxation that is not a true energy minimisation in a molecular force field;
- Contact anchors that are sparse and imperfect; hinge-based ϕ/ψ moves close distances only locally;
- Torsion training restricted to length ≤ 5, so medium-range motifs (e.g. β-turns spanning six residues) must emerge from assembly rather than from the torsion network.

4.3 Future work

Promising extensions that preserve PairFold's low-resource character include:

1. Distilling PLM embeddings (e.g. small ESM-2) into the contact tower for higher precision without full MSA pipelines;
2. Learning longer fragments (6–9-mers) or SE(3)-equivariant local frames while retaining DP segmentation;
3. Using contact maps to rank assembly hypotheses earlier in look-ahead search, not only in post-hoc hinge relax;
4. Finer sweeps of contact threshold, k, and temperature T with confidence–RMSD calibration on the Domains100 panel.

5 Conclusion

We have presented PairFold, a complete local backbone generation system that couples a fragment Transformer on (ϕ, ψ) with $O(N)$ segmentation, temperature-calibrated confidence, secondary-structure freezing, and look-ahead backtracking with lever-effect torsion relaxation. On 103 diverse domains the interactive consensus+SS path achieves mean inference time 2.67 s and mean Cα RMSD 25.88 Å; ablation on 20 domains shows the full local stack at ≈ 13.2 Å; and 100 tiled chains of length 10000–50000 complete in 0.36 s mean wall-clock. This illustrates a clear regime: **fast, resource-light, locally plausible backbones** rather than atomic-accuracy tertiary folds. In an era dominated by large folding networks, such local alternatives remain scientifically and practically valuable—especially when sequences grow long, hardware is modest, and the question of interest is local geometry augmented by a cheap contact prior.

Data and software availability

All software components described herein (`pairfold/predict.py`, `pairfold/clash_assembly.py`, `pairfold/structure_blocks.py`, and the `benchmarks/` suite) are part of the PairFold workspace. Expanded results are archived as `benchmark_domains100.csv`, `benchmark_long100.csv`, and `benchmark_ablation.csv`; tables and figures are regenerated by `python benchmarks/write_paper_ta` and `python benchmarks/plot_expanded.py`. Experimental structures were obtained from the RCSB PDB [10].

Acknowledgements

The authors thank the open structural biology community for publicly deposited high-resolution crystal structures that make fragment-level supervision possible, and the developers of PyTorch, BioPython, and the scientific Python stack.

References

[1] Jumper, J., Evans, R., Pritzel, A., et al. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature*, 596, 583–589. https://doi.org/10.1038/s41586-021-03819-2

[2] Senior, A. W., Evans, R., Jumper, J., et al. (2020). Improved protein structure prediction using potentials from deep learning. *Nature*, 577, 706–710. https://doi.org/10.1038/s41586-019-1923-7

[3] Lin, Z., Akin, H., Rao, R., et al. (2023). Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379, 1123–1130. https://doi.org/10.1126/science.ade2574

[4] Simons, K. T., Kooperberg, C., Huang, E., & Baker, D. (1997). Assembly of protein tertiary structures from fragments with similar local sequences using simulated annealing and Bayesian scoring functions. *Journal of Molecular Biology*, 268, 209–225. https://doi.org/10.1006/jmbi.1997.0959

[5] Rohl, C. A., Strauss, C. E. M., Misura, K. M. S., & Baker, D. (2004). Protein structure prediction using Rosetta. *Methods in Enzymology*, 383, 66–93. https://doi.org/10.1016/S0076-6879(04)83004-0

[6] Ramachandran, G. N., Ramakrishnan, C., & Sasisekharan, V. (1963). Stereochemistry of polypeptide chain configurations. *Journal of Molecular Biology*, 7, 95–99. https://doi.org/10.1016/S0022-2836(63)80023-6

[7] Engh, R. A., & Huber, R. (1991). Accurate bond and angle parameters for X-ray protein structure refinement. *Acta Crystallographica Section A*, 47, 392–400. https://doi.org/10.1107/S0108767391001071

[8] Berens, P. (2009). CircStat: A MATLAB toolbox for circular statistics. *Journal of Statistical Software*, 31, 1–21. https://doi.org/10.18637/jss.v031.i10

[9] Chou, P. Y., & Fasman, G. D. (1974). Prediction of protein conformation. *Biochemistry*, 13, 222–245. https://doi.org/10.1021/bi00699a002

[10] Berman, H. M., Westbrook, J., Feng, Z., et al. (2000). The Protein Data Bank. *Nucleic Acids Research*, 28, 235–242. https://doi.org/10.1093/nar/28.1.235